AstroChimp

Documentação do sistema

Projeto acadêmico | Turma A - manhã

Integrantes: Guilherme Lettieri, Lucas Mateus Andrade, Miguel Conceição Nunes e Pedro Gabriel de Oliveira.

Versão documentada: 27 de outubro de 2026.

1. Apresentação

O AstroChimp é um aplicativo de apoio à aula presencial. O professor cria uma sala, compartilha um código com a turma e acompanha as dúvidas e atividades pelo aplicativo. Os alunos participam com nome e código, sem precisar criar uma conta.

A proposta é reunir a comunicação da aula em um só lugar. Uma dúvida pode ser enviada por escrito e receber votos de colegas que também precisam daquela explicação. O professor pode destacar perguntas, compartilhar materiais e aplicar quizzes para acompanhar a compreensão do conteúdo.

1.1 Objetivo do projeto

O objetivo acadêmico é desenvolver um aplicativo móvel conectado a uma API e a um banco de dados. O projeto permite estudar telas, formulários, requisições HTTP, validação, autenticação e relacionamentos entre registros. A implementação utiliza React Native com TypeScript no aplicativo, Java com Spring Boot na API e MySQL para guardar os dados.

1.2 Perfis de uso

Perfil	Participação no sistema
Professor	Possui conta com e-mail e senha. Cria e encerra salas, acompanha dúvidas, compartilha materiais, controla o cronômetro e aplica quizzes.
Aluno	Entra em uma sala com nome e código. Envia dúvidas, vota em perguntas, consulta materiais e responde aos quizzes disponíveis.
Visitante	Consulta a apresentação pública do professor e os materiais que ele decidiu publicar.

1.3 Fluxo principal de uma aula

1. O professor entra no aplicativo e cria uma sala com o nome da aula.
2. O sistema fornece um código de seis caracteres para compartilhar com a turma.
3. Cada aluno informa seu nome e o código para entrar.
4. Durante a aula, a turma envia dúvidas, consulta conteúdos e participa das atividades.
5. O professor acompanha as perguntas, aplica quizzes e encerra a sala ao terminar.

As próximas seções apresentam as telas, regras, estrutura, dados, configuração e verificações desta versão.

2. Telas e navegação

O aplicativo possui 13 telas. A navegação é controlada pelo App.tsx, que guarda a sessão aberta e escolhe qual tela deve aparecer. Dados e ações são enviados aos componentes por propriedades, chamadas de props.

Tela	Finalidade
Início	Recebe nome e código para a entrada do aluno e oferece acesso à área do professor.
Login do professor	Recebe e-mail e senha e abre o painel após a autenticação.
Cadastro do professor	Cria a conta com os dados de identificação e o endereço do perfil público.
Painel do professor	Lista as salas do professor e permite criar uma nova aula ou abrir seu perfil.
Sala do professor	Mostra dúvidas, participantes e atividades. Reúne os controles de moderação, materiais, cronômetro, quiz e encerramento.
Perfil do professor	Permite editar a apresentação e organizar materiais e quizzes do próprio professor.
Perfil público	Exibe a apresentação do professor e os materiais marcados como públicos.
Sala do aluno	Mostra as dúvidas e os destaques da aula. Permite perguntar, votar e acessar materiais ou um quiz disponível.
Quiz do aluno	Apresenta a pergunta atual, as alternativas e o tempo restante para enviar uma resposta.
Materiais da sala	Lista os conteúdos compartilhados naquela aula. O professor também pode enviar e fixar materiais.
Criar material	Cadastra um link, um trecho de código ou um arquivo para compartilhar.
Escolher quiz	Lista os quizzes salvos e permite escolher uma atividade para aplicar à turma.
Criar quiz	Recebe título, perguntas, alternativas, resposta correta e tempo de cada pergunta.

2.1 Caminhos de uso

Professor: login ou cadastro, painel, sala e controles da aula. Pelo painel, também pode abrir o perfil e preparar conteúdos para usar depois.

Aluno: início, entrada por código e sala. A partir da sala, pode abrir a biblioteca de materiais e a tela do quiz.

Consulta pública: o perfil apresenta somente a descrição e os materiais publicados pelo professor. Os dados privados da conta ficam fora dessa consulta.

O login do professor e a participação do aluno ficam na memória do aplicativo enquanto a sessão está aberta. Ao sair ou recarregar o aplicativo, é necessário entrar novamente.

3. Regras da sala de aula

3.1 Criação, entrada e encerramento

A sala pertence ao professor que a criou. Seu código possui seis caracteres e permite a entrada dos alunos enquanto ela estiver ativa. Cada entrada registra um participante e gera um token, que identifica aquela participação nas próximas requisições.

Uma sala permanece ativa por até quatro horas. O professor pode encerrá-la antes desse prazo. Depois do encerramento ou da expiração, novas interações da aula são bloqueadas. O professor mantém acesso ao histórico registrado.

A quantidade de participantes mostra as entradas registradas na sala. Esse número não representa quantas pessoas estão conectadas naquele instante.

3.2 Envio de dúvidas e anonimato

O aluno pode escrever uma dúvida com até 280 caracteres e escolher se deseja aparecer como autor para os colegas. O texto precisa estar preenchido. A API identifica o aluno pelo token de participação e não aceita que ele escolha outro autor no pedido.

Quando a opção anônima está ativa, os outros alunos não recebem o nome do autor. O professor continua podendo identificar quem enviou a pergunta para acompanhar a turma.

3.3 Votação e organização das perguntas

Cada aluno pode votar uma vez em cada dúvida da sala. O voto indica que ele também gostaria de receber aquela explicação. O sistema confere a participação antes de registrar a escolha e recusa um segundo voto do mesmo aluno na mesma pergunta.

As dúvidas fixadas aparecem primeiro. As demais são organizadas pela quantidade de votos e pela data de criação, para ajudar o professor a localizar as perguntas mais relevantes.

O professor pode alterar o estado da dúvida:

Estado	Significado
Pendente	A pergunta ainda precisa ser acompanhada pelo professor.
Respondida	O professor já tratou da dúvida durante a aula.
Arquivada	A pergunta sai do mural ativo e permanece no histórico do professor.

3.4 Cronômetro de atividade

O professor informa o tempo de uma atividade, por exemplo, 120 segundos. O servidor registra quando a contagem começou e calcula o tempo restante nas consultas. Se passaram 30 segundos, a resposta informa que faltam 90.

Esse cronômetro é separado do prazo de quatro horas da sala. Ele pode ser iniciado ou parado pelo professor e chega a zero ao terminar a duração escolhida.

4. Materiais, perfil e quizzes

4.1 Biblioteca e compartilhamento de materiais

O professor pode cadastrar links, trechos de código e arquivos de até 10 MB. Um material pode ser preparado no perfil para uso posterior ou enviado durante a aula. Arquivos novos também podem ser enviados diretamente à sala.

Os materiais compartilhados ficam na biblioteca da aula. Quando o professor fixa um material, ele também aparece em destaque no mural. A fixação vale apenas para aquela sala; o mesmo conteúdo pode ser usado em outras aulas sem ficar fixado nelas.

Os arquivos são guardados na pasta de uploads do backend. O banco mantém os dados necessários para identificar o material. O aplicativo recebe a rota de download, sem conhecer o caminho interno do arquivo no computador do servidor.

4.2 Perfil do professor e consulta pública

O perfil reúne a apresentação do professor, sua biblioteca e os quizzes salvos. A edição permite atualizar dados de apresentação, como nome, descrição, foto e endereço público.

Um material só aparece no perfil público quando o professor o marca como público. Essa página não apresenta a senha nem o e-mail privado da conta. Publicar um material no perfil e compartilhá-lo em uma sala são ações diferentes.

4.3 Criação e aplicação do quiz

O professor prepara um quiz com título e perguntas. Cada pergunta tem enunciado, dificuldade, tempo para responder e de duas a seis alternativas. Exatamente uma alternativa deve estar marcada como correta, e a duração precisa ser maior que zero.

É possível salvar o quiz no perfil para reutilizar ou criar e aplicar uma atividade durante a aula. Quando um quiz é iniciado, o aluno vê que há uma atividade disponível e pode abrir a tela para responder à pergunta atual.

4.4 Respostas, tempo e resultados

O aluno escolhe uma alternativa e envia a resposta. A API aceita uma resposta por aluno, por pergunta e por rodada. Pedidos repetidos e respostas enviadas depois do tempo são recusados. A pergunta recebida pelo aluno não inclui o gabarito.

O professor consulta a quantidade de respostas por alternativa, avança para a próxima pergunta e encerra o quiz. O tempo é calculado a partir do momento em que a pergunta foi aberta no servidor.

Ao aplicar novamente o mesmo quiz, o sistema cria outra rodada. Os resultados da rodada nova começam do zero, preservando o registro das anteriores. Um quiz em uso ou com respostas registradas não pode ser excluído.

5. Arquitetura e organização

O sistema tem três partes: o aplicativo apresenta as telas, a API executa as regras e o MySQL guarda os registros. O aplicativo conversa com a API por requisições HTTP e dados em JSON.

Parte	Tecnologias e uso
Aplicativo	React Native, Expo SDK 57 e TypeScript. Componentes nativos e StyleSheet formam a interface.
Estado e comunicação	useState guarda dados das telas; useEffect organiza consultas e limpeza; fetch envia os pedidos à API.
API	Java 17, Spring Boot 4.1.1 e Maven. Controllers recebem os pedidos e services executam as regras.
Persistência	Spring Data JPA e Hibernate fazem a ligação entre as entidades Java e o MySQL.
Autenticação	Spring Security, HTTP Basic para o professor e token de participação para o aluno.
Verificação	JUnit e MockMvc na API, checagem de tipos do TypeScript e exportação pelo Expo.

5.1 Pastas do backend

Pasta	O que contém
controller	Rotas HTTP, parâmetros recebidos e respostas enviadas.
service	Validações, permissões e regras de cada funcionalidade.
repository	Consultas e gravações no banco de dados.
entity	Classes que representam os dados e seus relacionamentos.
dto	Campos selecionados para entrada ou saída de dados.
security	Configuração de autenticação e acesso às rotas.

5.2 Pastas do aplicativo

O arquivo App.tsx guarda a sessão e controla a tela aberta. A pasta src/pages contém as telas; src/components, os elementos reutilizados; src/services, os pedidos à API e o tratamento de arquivos; src/styles, os estilos compartilhados; src/types, os tipos dos dados; e assets, as imagens.

5.3 Exemplo de uma requisição

Ao enviar uma dúvida, a tela chama o serviço de API do aplicativo. O controller recebe o pedido, o service confere o aluno e a sala e o repository salva a pergunta no MySQL. A resposta retorna com os campos permitidos para quem fez a consulta.

Os services utilizam condições, laços e listas. No aplicativo, funções, arrays, props, useState e useEffect organizam os fluxos. As consultas da sala ocorrem a cada três segundos e são interrompidas ao sair da tela.

6. Dados do sistema

O banco principal se chama astrochimp. As entidades representam os registros usados pela aplicação. Algumas relações também geram tabelas auxiliares para ligar os dados.

Entidade	Dados e finalidade
Professor	Conta, senha em hash e apresentação. É o responsável pelas salas e pelos conteúdos que cria.
Sala	Nome da aula, código, professor, prazo de funcionamento, cronômetro e atividade de quiz atual.
Aluno	Nome e token de uma participação em uma sala. Não representa uma conta com senha.
Duvida	Texto, autor, opção de anonimato, estado, votos e fixação no mural.
Material	Título, tipo, conteúdo ou referência ao arquivo, descrição, publicação e professor responsável.
Quiz	Título, professor responsável, opção de salvar no perfil e lista de perguntas.
QuizPergunta	Enunciado, dificuldade, ordem, tempo para responder e alternativas.
QuizAlternativa	Texto da opção, letra e indicação de resposta correta.
QuizResposta	Aluno, alternativa escolhida e identificação da rodada em que respondeu.

6.1 Relações principais

Um professor pode ter várias salas, materiais e quizzes. Cada sala reúne seus participantes e dúvidas. Os materiais de um professor podem ser compartilhados em mais de uma sala.

Cada quiz possui perguntas, e cada pergunta possui alternativas. A resposta relaciona a escolha ao aluno e à rodada. Essa identificação separa as respostas quando a mesma atividade é aplicada novamente.

Os votos ligam alunos às dúvidas em que votaram. A fixação de materiais também pertence à sala, permitindo destacar um conteúdo em uma aula sem alterar sua apresentação nas outras.

6.2 Onde os dados ficam

Local	Conteúdo
MySQL: astrochimp	Cadastros, salas, participações, dúvidas, materiais e atividades.
backend/uploads	Arquivos enviados pelos professores.
Memória do aplicativo	Dados da sessão aberta, como o login do professor ou o token do aluno.
MySQL: astrochimp_test	Dados temporários usados pelos testes de integração.

O banco de testes é separado do banco de uso da aplicação. Suas tabelas são recriadas e removidas durante a suíte, por isso ele deve ser reservado aos testes.

7. Acesso e proteção dos dados

O professor utiliza e-mail e senha para autenticar seus pedidos. A senha da conta é armazenada como hash BCrypt, uma representação usada para conferir o login sem guardar a senha original. Ela não é devolvida nas respostas da API.

Depois do login, o servidor também confere se a sala, o material ou o quiz pertence ao professor. Estar autenticado não permite alterar os conteúdos de outro professor.

O aluno recebe um token ao entrar na sala. O aplicativo envia esse valor no cabeçalho X-Aluno-Token nas próximas requisições. A API confere se a participação pertence à sala consultada. Esse acesso não permite controlar as ações do professor.

7.1 Por que existem DTOs

Um DTO reúne os campos que uma operação precisa receber ou devolver. Isso ajuda a evitar que uma resposta exponha dados internos ou informações que aquele usuário não deve consultar. As validações de acesso continuam nos services.

A versão atual mantém 11 DTOs, agrupados abaixo por finalidade:

DTO ou grupo	Motivo de uso
PerfilProfessorDto	Recebe os dados de cadastro ou edição e apresenta o perfil sem devolver a senha.
PerfilPublicoProfessorDto	Apresenta dados públicos do professor e materiais publicados, sem e-mail privado nem senha.
SalaDto e EntradaSalaDto	Apresentam a aula e a participação que acabou de entrar, sem listar tokens dos outros alunos.
DuvidaDto	Devolve texto, votos e estado; a montagem da resposta preserva o anonimato para os colegas.
MaterialDto	Apresenta o conteúdo ou endereço de download, sem expor o caminho interno do arquivo.
CronometroDto	Informa o tempo escolhido para a atividade, os segundos restantes e se a contagem está ativa.
PerguntaAlunoDto e AlternativaAlunoDto	Mostram a pergunta e as opções ao aluno sem gabarito e representam a escolha enviada.
ResultadoPerguntaDto e ResultadoAlternativaDto	Reúnem as contagens de respostas por alternativa na rodada consultada pelo professor.

7.2 Credenciais do ambiente

A senha do MySQL pertence somente à configuração local do backend. Ela não deve aparecer no aplicativo, no repositório ou nesta documentação. O arquivo de ambiente do frontend informa apenas o endereço da API.

As credenciais da conta do professor e a senha do banco têm funções diferentes: uma identifica quem usa o aplicativo; a outra permite que o backend acesse o MySQL. Para uso fora do ambiente de desenvolvimento, a comunicação da API deve utilizar HTTPS.

8. Configuração e execução local

8.1 Preparar o ambiente

É necessário ter JDK 17 ou superior, Node.js e MySQL Server. O ambiente utilizado para o aplicativo usa Node.js 24. O Maven Wrapper já está no backend. O Workbench pode ser usado para consultar os registros, mas o serviço do MySQL Server precisa estar iniciado.

A configuração padrão utiliza o MySQL na porta 3306 e cria o banco astrochimp. O usuário do banco precisa ter permissão para criar o banco e as tabelas.

8.2 Configurar e iniciar o backend

Na raiz do projeto, abra um terminal e execute:

```
cd backend
Copy-Item application-local.example.properties application-local.properties
```

Abra backend/application-local.properties e preencha o usuário e a senha do seu MySQL. Esse arquivo é local e fica fora do Git. O arquivo público application.properties contém a configuração geral e carrega os valores locais.

Ainda na pasta backend, execute:

```
.\mvnw.cmd spring-boot:run
```

A API fica disponível em http://localhost:8080. As variáveis DB_URL, DB_USERNAME e DB_PASSWORD também podem fornecer a conexão quando não houver configuração local.

8.3 Instalar e abrir o aplicativo

Em outro terminal, a partir da raiz do projeto:

```
cd frontend
npm install
npm start
```

No celular, use uma versão do Expo Go compatível com o SDK 57 e leia o QR Code. Celular e computador precisam estar na mesma rede. O aplicativo usa o endereço do computador informado pelo Expo para encontrar a API na porta 8080.

Se a API estiver em outro endereço, copie frontend/.env.example para frontend/.env.local, ajuste EXPO_PUBLIC_API_URL e reinicie o Expo. No celular, localhost aponta para o próprio aparelho; use o endereço do computador que executa o backend.

8.4 Outras formas de abrir

npm run web abre a prévia no navegador. npm run android usa um emulador Android já configurado. npm run ios precisa de macOS para abrir o simulador. Mantenha o MySQL e a API em execução para utilizar os dados do sistema.

9. Verificação da versão

A verificação registrada em 27 de outubro de 2026 inclui 36 testes de integração da API com MySQL, sem falhas. Os casos cobrem cadastro e autenticação, permissões, salas, dúvidas, anonimato, votos, arquivos, cronômetro, rodadas de quiz e respostas duplicadas.

As 13 telas foram exercitadas na prévia do navegador, com sessões separadas de professor e aluno e integração com a API. Os fluxos incluíram entrada, criação de sala, dúvidas, votação, materiais, cronômetro, quiz e perfil. Essa verificação não substitui a execução do aplicativo em um aparelho.

9.1 Repetir as verificações

Na pasta backend, execute:

```
.\mvnw.cmd clean test
```

Os testes usam astrochimp_test com as credenciais do backend. As tabelas desse banco são recriadas e removidas pela suíte. Não utilize esse banco para guardar dados de aula.

Na pasta frontend, execute:

```
npm run typecheck
npx expo install --check
npx expo export --platform all
```

A checagem de tipos, a conferência de dependências e a exportação para Android, iOS e web passaram na versão verificada. A exportação gera pacotes de código; não instala nem executa o aplicativo em um celular.

10. Limitações e próximos passos

As telas acompanham a aula por consultas HTTP a cada três segundos. Ainda não há atualização por WebSocket, recuperação de senha ou manutenção da sessão depois de fechar e reabrir o aplicativo.

A execução em aparelho físico ou emulador ainda precisa ser validada, especialmente a seleção de arquivos, o compartilhamento, a cópia e a abertura de links. Os testes de pedidos simultâneos conferem duplicidade de votos e respostas; eles não representam um teste de carga com uma turma inteira.

Os próximos passos são concluir essa verificação no celular e avaliar melhorias a partir do uso em aula. O escopo atual concentra as atividades de uma sala; não inclui notas finais, frequência ou gestão completa de disciplinas.

11. Referências e documentação complementar

- README do projeto, documentação da API e documentação do aplicativo: configuração, telas e rotas desta implementação.
- React Native e Expo: componentes e execução do aplicativo.
- Spring Boot e MySQL: servidor e persistência de dados.